Московский авиационный институт (национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Информационный поиск»

Студент: П. А. Земсков
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б-23

Дата: 10.10.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №3 «Токенизация»

Необходимо написать модуль для разбиения текстов на токены с учётом специфики языка (знаки препинания, цифры, сокращения) и протестировать токенизатор на документах из корпуса.

- Токенизатор должен быть гибким и настроенным на конкретную предметную область — русскоязычные статьи.
- Нужно учесть нюансы: составные слова, аббревиатуры, числа, знаки препинания.
- Проверить работу на документах корпуса: количество токенов, их средняя длина, скорость работы.

1 Описание

1 Корпус

Токенизатор проверяется на корпусе первой лабораторной работы — 30 000 случайных статей русской Википедии (поле `text` в JSON-файлах, очищенный текст без таблиц и сносок). Объём текста — 231,2 МБ. Википедия хорошо подходит для проверки: в ней много сокращений («г.», «н. э.», «т. д.»), инициалов («А. С. Пушкин»), чисел всех видов (даты, десятичные дроби, числа с пробелами-разделителями), составных слов («Ростов-на-Дону», «северо-запад»), а также встречаются латиница, ссылки и адреса почты.

2 Общее устройство

Токенизатор — модуль `main.py` на Python 3 без внешних зависимостей (только стандартная библиотека). Вся логика — один регулярный шаблон с именованными группами и функция `tokenize(text)`, которая возвращает список пар `(вид, токен)`. Шаблон просматривает текст слева направо, на каждой позиции пробует варианты *в порядке приоритета* и берёт первый подошедший; поэтому более специфичные правила (ссылка, инициалы, сокращение, число) стоят раньше общих (слово). Всё, что не подошло ни к одному правилу (пробелы, подчёркивание), пропускается.

3 Правила токенизации

Вид	**Правило**	**Примеры**
`url`, `email`	`http(s)://` или `www.` до пробела; знаки препинания в конце не входят в ссылку; адрес почты	`www.eol.org`, `https://doi.org/10.1/x`
`initials`	заглавная буква с точкой (одна или две подряд) перед фамилией; римские цифры `I`, `V`, `X` исключены, иначе «Пётр I. Он» склеивается	«А. С. Пушкин» → `a.c.`, `пушкин`
`abbr`	«т. д.», «т. е.», «н. э.»; цепочки «буква-точка»; слова из словаря с точкой («г.», «гг.», «руб.», «англ.», …); «в.»/«г.» после числа или римской цифры	`т.е.`, `н.э.`, `г.`, `руб.`, `XX` + `в.`

`number`	целые, десятичные (`3,14`), даты (`12.03.2020`), время (`10:30`); группы по три цифры через пробел (`10 000`)	`3,14`, `10 000`
`compound`	буквы и цифры через дефис; диапазон из одних цифр (`1990-2000`) разбивается на два числа	`ростов-на-дону`, `кто-то`, `f-16`, `1990-е`
`word`	последовательность букв и цифр, апостроф между буквами не разрывает слово	`пушкин`, `o'neil`, `3d`
`punct`	любой другой знак; по умолчанию отбрасывается, возвращается при `punct=True`	`,` `.` `(`

4 Нормализация

- Приведение к нижнему регистру (`lower=True`), замена «ё» на «е» (`yo=True`): в Википедии одно и то же слово пишется обоими способами, и без замены оно распадается на два термина.
- Удаляются мягкий перенос (U+00AD) и знак ударения (U+0301): слово с ударением на второй букве («Глау») иначе распадается на два токена.
- Пробелы внутри сокращений убираются: «т. е.» → `т.е.`
- Составные слова по умолчанию остаются одним токеном; параметр `split_compounds=True` разбивает их на части («ростов», «на», «дону») — это нужно, если поиск по части слова важнее точного совпадения.

5 Запуск

```
python main.py # whole corpus, statistics
python main.py --limit 1000 # first 1000 documents
python main.py --text "text to tokenize"
python test_main.py # unit checks
```

6 Пример работы

Предложение: «В 1823 г. А. С. Пушкин жил в Ростове-на-Дону, т. е. на юге: 10 000 руб., 3,14% на 12.03.2020.» Сравнение с наивным разбиением по `\w+` (всё, что не буква

и не цифра, — граница):

Метод	**Токены**
`\w+` (24)	в, 1823, г, а, с, пушкин, жил, в, ростове, на, дону, т, е, на, юге, 10, 000, руб, 3, 14, на, 12, 03, 2020
Токенизатор (16)	в, 1823, г., а.с., пушкин, жил, в, ростове-на-дону, т.е., на, юге, 10 000, руб., 3,14, на, 12.03.2020

Наивный метод рвёт «а. с.», «т. е.» на одиночные буквы (шум: токены «а», «с», «т», «е» совпадают с предлогами и союзами), режет «10 000» и дату на несколько чисел, а «3,14» превращает в два числа. Токенизатор сохраняет каждую из этих единиц как один токен.

7 Результаты на корпусе

Прогон по всем 30 000 документам (один поток, Python 3.13):

Показатель	**Значение**
Документов / объём текста	30 000 / 231,2 МБ
Токенов (наш токенизатор)	17 765 479
Токенов (`\w+`)	18 287 320
Токенов (с `split_compounds`)	18 072 387
Уникальных токенов (наш / `\w+` / с разбиением)	978 389 / 896 072 / 912 413
Токенов, встречающихся один раз (наш)	519 230 (53%)
Средняя длина токена	6,08 символа
Время токенизации	35,5 с
Скорость	6,2 МБ/с (6368 КБ/с), 500 тыс. токенов/с

Распределение по видам токенов:

Вид	**Токенов**	**Доля**
`word`	16 006 198	90,2%
`number`	1 231 352	6,9%
`compound`	291 642	1,6%
`abbr`	112 483	0,6%
`initials`	95 304	0,5%
`url`	2 429	<0,1%
`email`	9	<0,1%

Распределение по длине токена: 1 символ — 11,0%, 2–3 — 14,5%, 4–7 — 42,4%, 8 и больше — 32,1%.

Самые частые токены разных видов:

- составные: «из-за», «1-й», «2-й», «санкт-петербург», «генерал-майор», «нью-йорк»;
- сокращения: «англ.», «г.», «см.», «гг.», «ред.», «н.э.»;
- инициалы: «а.а.», «в.а.», «в.в.», «в.и.»;
- ссылки: «www.youtube.com», «www.sports-reference.com».

8 Тестирование

Файл `test_main.py` содержит 18 проверок правил (пунктуация, «ё», ударение, составные слова, тире с пробелами, диапазоны, числа с разделителями, сокращения, инициалы, римские цифры, ссылки и почта) и проверки параметров `split_compounds`, `lower`, `punct`. Все проверки проходят (`python test_main.py` печатает `ok`).

2 Исходный код

Программа `main.py` состоит из регулярного шаблона `PATTERN`, функции `tokenize` (токенизация и нормализация), функции `baseline` (наивное разбиение для сравнения) и функции `run` (прогон по корпусу и статистика). Приведён упрощённый шаблон (кириллические классы символов заменены на латинские для компактности; адрес почты опущен) и основная функция.

```
PATTERN = re.compile(r"""
    (?P<url>(?:https?://|www\.)[^\s<>"'\])]*[^\s<>"'\]).,;:!?]) |
    (?P<initials>[A-HJ-UW-Z]\.\s?(?:[A-HJ-UW-Z]\.\s?)?(?=[A-Z][a-z])) |
    (?P<abbr>
        (?i:(?:t\.\s?[dpe]|n\.\s?e)\.) | # "t. d.", "n. e."
        (?:[^\W\d_]\.){2,} | # dotted: "t.d.", "e.g."
        (?:(?<=[\dIVX][ \xa0])|(?<=[\dIVX]))[vg]\. | # "XX v.", "1990 g."
        (?i:(?:%s)\.) # dictionary of abbreviations
    ) |
    (?P<number>
        (?<!\d)\d{1,3}(?:[ \xa0\u202f\u2009]\d{3}(?!\d))+(?![\w-]) | # 10 000
        \d+(?:[.,:/]\d+)*(?!\w|-\w) # 3,14 12.03.2020
    ) |
    (?P<compound>[^\W_]+(?:[-\u2010\u2011][^\W_]+)+) | # word-word, F-16, 1990-e
    (?P<word>[^\W_]+(?:['\u2019][^\W_]+)*) | # words, O'Neil, 3D
    (?P<punct>[^\w\s])
""" % "|".join(ABBR), re.VERBOSE)

DROP = str.maketrans("", "", "\u00ad\u0301") # soft hyphen, stress mark

def tokenize(text, lower=True, yo=True, punct=False, split_compounds=False):
    out = []
    for m in PATTERN.finditer(text.translate(DROP)):
        kind, tok = m.lastgroup, m.group()
        if kind == "punct" and not punct:
            continue
        if kind in ("abbr", "initials"):
            tok = re.sub(r"\s", "", tok) # "t. e." -> "t.e."
        if kind != "url" and kind != "email":
            tok = tok.lower() if lower else tok
            tok = tok.replace("\u0451", "e") if yo else tok
        if kind == "compound" and (split_compounds
                or re.fullmatch(r"[\d\-\u2010\u2011]+", tok)):
            # numeric ranges are never one token
            out += [("number" if p.isdigit() else "word", p)
                    for p in re.split("[-\u2010\u2011]", tok)]
        else:
            out.append((kind, tok))
    return out
```

В реальном коде классы символов и словарь сокращений содержат кириллицу (`А-ЯЁ`,

т, д и т. д.), а «ё» записана буквой; в листинге это заменено на латиницу и код `\u0451`, так как `listings` плохо работает с кириллицей.

3 Выводы

Выполнив третью лабораторную работу по курсу «Информационный поиск», я написал токенизатор для русскоязычных текстов и проверил его на корпусе из 30 000 статей Википедии. Основное, что я узнал нового:

- токенизация — не «разбить по пробелам»: наивное разбиение по не-буквам даёт 18,29 млн токенов, а токенизатор с правилами — 17,77 млн. Разница в 0,5 млн токенов — это числа, даты, сокращения, инициалы и составные слова, которые наивный метод рвёт на мусор из одиночных букв и цифр;
- порядок правил важен: регулярное выражение берёт первый подходящий вариант, поэтому ссылка, инициалы, сокращение и число должны стоять раньше слова, а число — проверять, что за ним не идёт буква или дефис, иначе «3D» и «1990-е» распадаются;
- неоднозначность решается эвристиками: точка после «г» — сокращение, но «в» — предлог, поэтому «в.» признаётся сокращением только после числа или римской цифры; «Пётр I. Он» нельзя принять за инициалы, поэтому римские цифры исключены; десятичную запятую (`3,14`) и разделитель тысяч (`10 000`) нужно отличать от списка чисел, поэтому группа обязана состоять ровно из трёх цифр;
- нормализация данных не менее важна, чем разбиение: без замены «ё» на «е» и удаления знака ударения одно слово становится двумя терминами или распадается на части;
- составные слова — вопрос выбора: цельный токен («санкт-петербург») точнее, но словарь больше (978 тыс. против 912 тыс. уникальных токенов при разбиении), поэтому разбиение вынесено в параметр;
- числа раздувают словарь: 53% различных токенов встречаются ровно один раз (519 тыс.) — в основном даты, десятичные дроби и номера; в индексе их, вероятно, стоит нормализовать или не индексировать;
- скорость регулярного выражения достаточная: 6,2 МБ/с (около 500 тыс. токенов в секунду) на одном потоке, весь корпус обработан за 35,5 с.

Известные ограничения: словарь сокращений конечен (незнакомое сокращение с точкой остаётся словом, а точка отбрасывается); «г.» всегда считается сокращением, даже в конце предложения; одиночная заглавная буква с точкой перед словом с заглавной буквы считается инициалом; не обрабатываются `C++`, `C#` и другие символьные названия. Для задач поиска этого достаточно; полученный токенизатор будет использоваться в следующих лабораторных работах при построении индекса.

Список литературы

[1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Клюшина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))

[2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008

[3] Русская Википедия // https://ru.wikipedia.org

[4] Python Documentation: re — Regular expression operations // https://docs.python.org/3/library/re.html

[5] Unicode Standard Annex #29: Unicode Text Segmentation // https://unicode.org/reports/tr29/